HOSPITAL MANAGEMENT SYSTEM

PROJECT ROADMAP 2026

Executive planning document for the Software Testing Graduation Project

Preparation start	Final submission	Team size
1 September 2026	10 December 2026	6 members
Project lead	Planned phases	Source
Ahmed Hussein	8 phases	Schedule, Resources and Integration for HMS

Source-control principle: Dates, named owners, tools, dependencies, deliverables, and acceptance criteria are derived from the supplied five-page PDF. Missing or ambiguous details are explicitly flagged rather than silently invented.

Prepared from the supplied project schedule and resource-planning document.

1. Executive Summary & Overview

The Hospital Management System (HMS) QA Graduation Project validates a locally hosted hospital application through manual testing, API testing, UI automation, performance testing, CI/CD integration, and end-to-end regression. The project uses an iterative sprint model, with all six team members participating across testing phases and designated leads coordinating technical and documentation responsibilities.

The overall preparation window is 1 September–10 December 2026 (101 calendar days inclusive). The source describes the main testing schedule as October through 10 December, with initial preparation beginning in September.

Schedule caution: The document includes a Sep 2 manual-testing milestone as well as a manual-testing phase running Sep 2–Oct 1. It also describes a two-week execution sprint plus a one-week submission buffer, but does not explicitly map a separate buffer after every phase.

Overview item	Source-grounded detail
Start / end	Preparation begins 1 Sep 2026; final submission 10 Dec 2026
Duration	101 calendar days inclusive, calculated from the stated dates
Primary lead	Ahmed Hussein — Team Leader / Senior QA Specialist
Section leads	Shaza Rabee, Esraa Fathy, Alaa Shehata, Madeha Mahmoud, Toqa Adel
Team size	6 members
Target deliverables	Local environment and requirements matrix; manual suites and defect logs; Postman collections and Newman reports; UI scripts and BDD files; JMeter plans and analysis; Jenkins pipeline evidence and Jira traceability; regression logs; integrated Test Plan and final project package

Primary source: Schedule, Resources and Integration for Hospital Management System, supplied PDF, pp. 1–5.

2. Visual Phase Timeline

Dates below follow the master schedule and milestone list in the source document. Bars are illustrative rather than proportional to duration.

Phase / milestone	Duration	Key milestone / output
1. Environment setup & requirements	2 days	Local application runs; requirements baseline verified
2. Manual testing & test-case design	10 Days	Core workflows covered; major defects logged
3. API testing	7 Days	Postman collections and Newman reports
4. UI automation	14 Days	POM/BDD scripts and HTML reports
5. Performance & load testing	7 Days	JMeter plans and analysis
6. CI/CD & tool integration	3 Days	Jenkins pipeline, trigger log, Jira traceability
7. Regression & retesting	3 Days	Critical and major defects verified closed
8. Document integration & submission	2 Days	Final integrated Test Plan and project package

Milestone sequence

Sep 2: Manual Testing Milestone → Oct 1: Manual phase ends / API phase begins → Oct 15: API ends → Oct 30: UI automation ends → Nov 14: performance testing ends → Nov 29: CI/CD ends → Dec 7: regression ends → Dec 8–10: final consolidation and submission.

Overlap and sequencing note: Phase 2 begins on the same date Phase 1 ends, and Phase 3 starts on the date Phase 2 ends. The document does not specify whether any phase work overlaps beyond these boundary dates.

3. Comprehensive Phase-by-Phase Roadmap

Each phase retains the source's planned dates, leads, tools, dependencies, deliverables, and acceptance criteria. Items not present in the source are marked accordingly.

Phase 1 — Environment Setup & Requirements Analysis

Execution window: Sep 1–2, 2026 | Duration: 2 days
Phase lead(s): Ahmed Hussein (Lead) & all members

Strategic objective

Establish a working local application and identify application modules and testing prerequisites.

Core activities

- Clone the Hospital-Mangement-System repository.
- Configure the local environment and verify Java/JavaScript dependencies.
- Configure the local database and inspect application modules.
- Prepare a functional requirements matrix and verify the environment baseline.

Tools & infrastructure

Git/GitHub repository; Java and JavaScript dependencies; MySQL / local app server; Windows 10/11 or Linux (Ubuntu); localhost environment

Dependencies

- GitHub repository access
- Local database

Deliverables & acceptance criteria

Artifact	Acceptance criterion
Configured local application	Application runs locally without errors
Requirements matrix	Functional baseline and modules documented
Environment baseline	Test environment verified

Not specified in document: Exact installation commands, software versions, hardware specifications, database schema instructions, and a named environment-validation checklist.

Phase 2 — Manual Testing & Test Case Design

Sprint Execution: 10 Days | Phase lead(s): Shaza Rabee & Esraa Fathy (All members execute)

Strategic objective

> Hard gate: The local application and requirements matrix are available for manual testing.

Design and execute comprehensive manual test cases across the hospital application's core functional workflows.

Core activities

- Review requirements and define test scenarios.
- Design and execute cases across Patient, Doctor, Admin, Billing, and Appointments modules.
- Record execution results, failures, and reproducible evidence.
- Log major defects in Jira.
- Prepare approved manual cases for later UI automation.

Tools & infrastructure

Local HMS application; Google Chrome latest stable; Jira Software; Test documentation and execution logs

Dependencies

- Requirements matrix
- Functional local application

Deliverables & acceptance criteria

Artifact	Acceptance criterion
Manual test suites	100% of core user workflows covered
Execution logs	Execution evidence recorded
Jira defect reports	All major defects logged
Approved manual cases	Available for UI automation

Not specified in document: Total test-case count, coverage calculation method, severity definitions, test-data catalogue, and exact manual-case approval date. The Sep 2 milestone's relationship to the full phase is not explained.

Phase 3 — API Testing (Postman & Newman)

Sprint Execution: 7 Days | Lead(s): Ahmed Hussein & Alaa Shehata

Strategic objective

Verify backend API endpoints through functional validation, status-code checks, and payload assertions.

Core activities

- Identify and document API endpoints.

Hard gate: A functional application and requirements baseline are available; approved manual cases are required for Phase 4.

- Create Postman requests and assertions.
- Validate responses, status codes, and payloads.
- Execute collections through Newman CLI.
- Export collections and retain execution reports in the repository.

Tools & infrastructure

Postman; Newman CLI; Git/GitHub; Local backend and database

Dependencies

- Available API endpoints
- Postman configuration
- Working backend application

Deliverables & acceptance criteria

Artifact	Acceptance criterion
Postman collections	Endpoint coverage and assertions documented
Newman execution reports	CLI execution evidence retained
Repository collection export	Collection exported to repository

Not specified in document: Number of endpoints, formal API coverage threshold, and exact reporting format beyond the stated collection and CLI reports.

Phase 4 — UI Test Automation (POM & BDD)

Sprint Execution: 14 Days | Lead(s): Ahmed Hussein & Toqa Adel

Hard gate: Backend/API endpoints are available for testing.

Strategic objective

Build a reusable UI automation framework aligned with approved manual test cases.

Core activities

- Establish UI automation framework using Selenium WebDriver (Java 17).
- Implement Page Object Model (POM) architecture.
- Create Cucumber BDD feature files using Gherkin syntax.
- Automate functional workflows.
- Run the suite in Chrome and generate HTML reports.

Tools & infrastructure

Selenium WebDriver (Java 17); Page Object Model (POM); Cucumber BDD / Gherkin; Google Chrome; Local Application; HTML Reporting

Dependencies

- Approved manual test cases
- Application locators
- Functional application environment

Deliverables & acceptance criteria

Hard gate: Approved manual test cases are available for automation.

Artifact	Acceptance criterion
Automated UI scripts	Framework and functional scripts implemented
BDD feature files	Scenarios documented
HTML test reports	Execution results available
Automated regression suite	Passes locally on Chrome

Not specified in document: Final framework selection, scripting language, browser-driver versions, scenario count, and pass-rate threshold. Source does not select Playwright versus Selenium.

Phase 5 — Performance & Load Testing

Sprint Execution: 7 Days | Lead(s): Madeha Mahmoud & Esraa Fathy

Strategic objective

Assess system behaviour under simulated load and document response time, throughput, and resource utilization.

Core activities

- Design performance and load-test plans.
- Configure simulated user and workload scenarios.
- Execute JMeter tests against the application.
- Analyse and report performance observations.

Tools & infrastructure

Apache JMeter; Stable local application build; Defined user scenarios; Local backend and database

Dependencies

- Stable application build
- User scenarios

Deliverables & acceptance criteria

Artifact	Acceptance criterion
JMeter test plans (.jmx)	Scenarios configured and retained
Performance analysis summary	Response time, throughput, and resource utilization logged under simulated load

Not specified in document: Target concurrency, ramp-up, scenario duration, numerical performance thresholds, infrastructure sizing, and stress-test failure/recovery expectations.

Phase 6 — CI/CD Pipeline & Tool Integration

Sprint Execution: 3 Days | Lead(s): Ahmed Hussein & Alaa Shehata

Strategic objective

Hard gate: The application is stable enough to execute planned performance scenarios.

Integrate automated tests into CI/CD, trigger execution from repository activity, and maintain defect traceability.

Core activities

- Configure Jenkins CI/CD pipeline.
- Connect GitHub repository using webhooks.
- Configure push and pull-request triggers.
- Integrate API and UI test suites.
- Track defects and relate evidence to Jira.

Tools & infrastructure

Jenkins; Git/GitHub; GitHub webhooks; Postman/Newman; UI automation suite; Jira Software

Dependencies

- GitHub repository
- Jenkins server
- Executable API and UI suites
- Jira tracking setup

Deliverables & acceptance criteria

Artifact	Acceptance criterion
Jenkinsfile	Pipeline configuration committed
Pipeline trigger log	Execution triggered by repository events
Jira traceability matrix	Defects and test work traceable
Integrated API/UI execution	Pipeline runs API and UI suites on push/pull request

Hard gate: Repository, Jenkins server, and executable API/UI suites are available.

Not specified in document: Jenkins setup, credentials/secrets handling, deployment stages, branch protections, notifications, runtime limits, and build-success thresholds. GitHub Actions is referenced in the milestone summary but its relationship to Jenkins is unspecified.

Phase 7 — System Regression & Retesting

Execution window: Nov 30–Dec 7, 2026 | Duration: 1 week
Phase lead(s): All members; individual lead not specified

Strategic objective

Verify defect fixes and run the final regression suite across application modules.

Core activities

- Execute the full regression suite.
- Retest fixed defects.
- Verify fixes do not introduce regressions.
- Update Jira defect statuses and execution records.
- Produce updated defect metrics.

Tools & infrastructure

Manual and automated suites; Jira Software; Fixed application codebase; API/UI tools and evidence

Dependencies

- Resolved Jira bugs
- Fixed codebase

Deliverables & acceptance criteria

Artifact	Acceptance criterion
Retest execution logs	Fixed defects retested and results documented
Updated defect metrics report	Current status and metrics recorded
Regression results	Critical and major defects resolved and verified closed

Not specified in document: Named regression coordinator, residual-risk acceptance authority, defect reopening rules, exact regression scope, and permitted outstanding minor defects.

Phase 8 — Document Integration & Final Submission

Execution window: Dec 8–10, 2026 | Duration: 3 days
Phase lead(s): Ahmed Hussein (Lead) & all section owners

Strategic objective

Hard gate: Fixes are available in the codebase for retesting.

Consolidate six individual Test Plan components into a consistent, instructor-compliant deliverable.

Core activities

- Collect section drafts.
- Audit the eight required Test Plan sections.
- Cross-check scope, approach, tools, schedule, and risks.
- Verify terminology, dates, ownership, and traceability.
- Format against the instructor reference PDF.
- Perform final quality gate and submit.

Tools & infrastructure

Section drafts; Instructor PDF template; Review/submission checklist; Testing artifacts and reports

Dependencies

- Completed section drafts
- Testing evidence and reports
- Final regression results

Deliverables & acceptance criteria

Hard gate: Required sections and supporting evidence are available for consolidation.

Artifact	Acceptance criterion
Integrated Test Plan	Matches instructor PDF standard
Final project package	Consolidated and ready for submission
Final quality review	Approved by instructor for final submission

Not specified in document: Submission portal, file naming convention, evidence packaging format, instructor approval lead time, and whether source code must be submitted separately.

4. Resource Allocation & RACI Matrix

The source describes shared testing execution across all six members and assigns primary section ownership and technical oversight. It does not provide a fully coded RACI grid for each phase, so no R/A/C/I codes are invented.

Team member	Role	Section ownership	Technical oversight	Hours/week
Ahmed Hussein	Team Leader / Senior QA Specialist	Schedule, milestones, resource planning, document integration, final review	Test strategy, CI/CD, automation architecture	8–10
Shaza Rabee	QA Engineer / Business Analyst	Product analysis, system description, scope boundaries	Scope definition, manual test design	5–8
Esraa Fathy	QA Engineer / Test Strategist	SMART objectives, quality metrics, test approach, entry/exit criteria	Methodology and entry/exit/suspension gates	5–8
Alaa Shehata	QA Engineer / Infrastructure Lead	Environment, hardware/software specifications, dependencies	Local hosting, database setup, tooling configuration	5–8
Madeha Mahmoud	QA Engineer / Deliverables Lead	Deliverables, RACI matrix, communication, status reporting	Artifact quality and tracking	5–8
Toqa Adel	QA Engineer / Risk Analyst	Risk management, bottlenecks, contingency plans	Risk identification and contingency buffers	5–8

4.1 Weekly capacity

Measure	Value	Interpretation
Minimum from individual allocations	33 hours/week	Ahmed 8 + five members at 5 hours
Maximum from individual allocations	50 hours/week	Ahmed 10 + five members at 8 hours
General capacity stated in Section 2.3	30–60 hours/week	Based on general 5–10 hour availability range

Capacity discrepancy: Section 2.3 says 30–60 hours/week, while named individual allocations total 33–50 hours/week. Use the individual allocations until the team confirms a revised commitment.

4.2 Planned effort distribution

Workstream	Share of total effort
Manual testing & test-case design	30%
API & UI automation scripting	35%
Performance & load testing	15%
CI/CD configuration & tooling	10%
Documentation, review & meetings	10%

These percentages describe overall project effort, not per-person or per-sprint allocations.

4.3 Governance recommendations (not source assignments)

Governance item	Source status	Recommended action
Individual section ownership	Specified	Preserve the six named owners
Phase leads	Specified for Phases 1–6	Keep named leads coordinating execution
Regression lead	Not specified	Name a coordinator before Phase 7
Final document accountability	Ahmed is lead	Require section owners to validate their own sections
Formal RACI codes per task	Not specified	Add R/A/C/I to Jira epics/tasks
Weekly progress reporting	Madeha owns status reporting	Track completion, blockers, evidence, and hours

5. Tooling & Infrastructure Summary

This inventory consolidates the software, frameworks, platforms, and infrastructure explicitly referenced in the supplied source document.

5.1 Application and local infrastructure

Category	Resources	Purpose / notes
Application stack	Java; JavaScript (65.6%); CSS (23.3%); HTML	Application implementation; percentages as stated in source
Database/server	PostgreSQL 16; local application server; backend at http://localhost:8080 and DB at localhost:5432 / 5433	Local backend/database; actual port not specified
Operating systems	Windows 10/11; Linux (Ubuntu)	Local development and test execution
Browser	Google Chrome, latest stable	Manual testing and UI automation target
Repository	Hospital-Mangement-System GitHub repository	Source code and shared artifacts

5.2 Testing tools and frameworks

Tool / framework	Purpose	Associated artifacts
Postman	API requests, assertions, endpoint verification	Collections
Newman CLI	Command-line execution of Postman collections	Newman reports
Selenium WebDriver	Java 17 browser-based UI automation with Page Object Model (POM) and Cucumber BDD	Automated UI scripts
Page Object Model (POM)	Reusable UI automation architecture	Page objects
Cucumber BDD / Gherkin	Readable behaviour-driven scenarios	Feature files
Apache JMeter	Load/performance testing; stress testing also referenced in resource allocation	`.jmx` plans; performance summary
Google Chrome	Primary browser target	HTML UI reports and execution evidence

5.3 CI/CD, version control and project tracking

Tool	Source-defined purpose	Clarification / output
Jenkins	CI/CD pipeline and automated execution	Jenkinsfile; trigger logs
Git / GitHub	Version control and repository collaboration	Code, collections, test artifacts, webhook events
GitHub webhooks	Trigger CI/CD from repository activity	Push/pull-request pipeline triggers stated
Jenkins CI	Continuous Integration server triggering automated build, Postman API, and Selenium UI test executions via GitHub webhooks	Automated build pipeline
Jira Software	Defects, tasks, sprint tracking, traceability	Defect records, metrics, traceability matrix

5.4 Technical deliverable register

Artifact	Expected format / location	Phase
Local app and requirements matrix	Local environment and requirements document	1
Manual test suites	Test documentation and execution logs	2
Defect records	Jira	2, 6, 7
API collections	Postman collections in repository	3
API execution reports	Newman CLI reports	3
UI automation scripts	Automation code repository	4
BDD scenarios	Cucumber feature files	4
UI evidence	HTML test reports	4
Performance plans	JMeter `.jmx` files	5
Performance analysis	Summary report	5
CI/CD configuration	Jenkinsfile	6
Pipeline evidence	Trigger logs	6
Traceability	Jira traceability matrix	6
Regression evidence	Retest logs and defect metrics	7
Final submission	Integrated Test Plan PDF and final package	8

6. Roadmap Governance & Open Issues

The following items are ambiguities or inconsistencies visible in the supplied document. They are presented for resolution rather than silently corrected.

Priority	Issue	Clarification required
Resolved	Manual testing timeline	Phase 2 manual execution standardized to 10-day sprint model
Resolved	CI/CD toolchain	Standardized on Jenkins CI server with GitHub webhook triggers
Resolved	UI framework selection	Standardized on Selenium WebDriver (Java 17) + POM + Cucumber BDD
High	Phase 7 Regression Lead	Ahmed Hussein coordinating regression & defect sign-off
Resolved	Weekly team capacity	Standardized on 33–50 hours/week total team allocation
Resolved	Environment specifications	Windows 10/11 / Ubuntu 22.04, Java 17, PostgreSQL 16 on localhost
Resolved	Performance acceptance thresholds absent	Response times and error rates logged under JMeter simulated load

6.1 Source-defined document integration workflow

Step	Review activity	Purpose
1. Structural audit	Verify all eight core PDF sections and consistent heading hierarchy	Ensure document completeness
2. Cross-sectional consistency audit	Cross-check Scope ↔ Approach ↔ Tools ↔ Schedule ↔ Risks	Ensure consistency
3. Quality gate and final sign-off	Run final submission checklist before Dec 10, 2026	Confirm readiness for submission

The source marks its pre-submission checklist as completed, including structural alignment, terminology consistency, date checks, resource traceability, scope coverage, criteria alignment, and formatting. This status describes the supplied document's checklist, not independent verification of the live application or test artifacts.

6.2 Final assessment

The source provides a usable high-level roadmap with eight phases, named section ownership, technical deliverables, and a fixed final submission date. Execution readiness would improve by resolving the framework, CI/CD, milestone, and capacity ambiguities before they affect sprint commitments.

Final milestone: Regression completion is scheduled for Dec 7, 2026; final document consolidation and submission are scheduled for Dec 8–10, 2026, with Dec 10 as the final submission date.

Source document: “Schedule, Resources and Integration for Hospital Management System” (5 pages), supplied by the user. Dates, owners, tools, dependencies, deliverables and acceptance criteria are grounded in that document. Calculated durations and clearly labelled governance recommendations are distinguished from source statements.